FC Transaction Management System

Technical Documentation

Version 1.0.0 | March 2026

Node.js · Express.js · React · Vite · CSV Storage

Monorepo Architecture via npm workspaces

1. System Overview

FC Transaction Management System is a full-stack financial transaction management application. It provides a REST API backend for data operations and a React frontend for displaying and adding transactions. All transaction data is persisted in a CSV file, making it lightweight and portable without requiring a database server.

Key Features

- Partial CRUD operations — create and read financial transactions
- Filter transactions by status, date range, and search by account holder name and number
- Shared validation package enforced on both frontend and backend
- Monorepo architecture using npm workspaces for clean code organization

Tech Stack

Layer	Technology	Purpose
Backend	Node.js, Express.js	REST API, CSV file operations, PDF generation
Frontend	React, Vite	Single-page application, dashboard, transaction UI
Storage	CSV file	Lightweight persistent data — no database required
Validation	@transaction/shared	Shared rules used by both frontend and backend

Layer	Technology	Purpose
Architecture	npm workspaces	Monorepo managing all three packages from one root

2. Prerequisites

Ensure the following software is installed before setting up the project:

Software	Minimum Version	Verify With	Download
Node.js	v18.0.0	node --version	nodejs.org
npm	v7.0.0	npm --version	Included with Node.js
Git	Any recent version	git --version	git-scm.com

> Note: npm v7+ is required for npm workspaces support. It ships automatically with Node.js v15+. To upgrade: npm install -g npm

3. Installation

Follow these steps in order. All commands are run in your terminal or command prompt.

Step 1 — Clone the Repository

```
git clone https://github.com/axvolkzki/FC-transaction-management-system.git
cd FC-transaction-management-system
```

Step 2 — Install All Dependencies

```
npm install
```

> Note: A single npm install from the root folder installs all dependencies for the backend, frontend, and shared package simultaneously. No need to run it inside any subfolder.

Step 3 — Setup Initial Sample Data

```
npm run setup
```

> Note: npm run setup does two things: (1) copies transactions.sample.csv to transactions.csv as the starting dataset, and (2) runs migrate.js to assign a cryptographically secure UUID v4 to every existing record. Only needs to be run once on first setup.

Step 4 — Run the Application

```
npm run dev
```

Open your browser and go to http://localhost:3000 to access the application.

4. Configuration

No environment variables are required. Configuration is handled through these files:

File	Purpose	Key Settings
backend/server.js	API server configuration	PORT = 3001, CSV file path
frontend/vite.config.js	Frontend dev server	PORT = 3000, /api proxy to port 3001
shared/validation.js	Shared validation rules	VALID_STATUSES, ACCOUNT_NUMBER_REGEX
package.json (root)	Monorepo and scripts	Workspace definitions, dev/setup scripts

Changing Ports (Optional)

Backend — edit backend/server.js:

```
const PORT = 3001;   // Change to any available port
```

Frontend — edit frontend/vite.config.js:

```
server: { port: 3000, proxy: { '/api': { target: 'http://localhost:3001' } } }
```

> Warning: If you change the backend port, update the proxy target in vite.config.js to match. Both values must be identical.

5. Running the Application

Option A — Both Servers at Once (Recommended)

```
npm run dev
```

Starts both servers simultaneously. Expected terminal output:

```
[0] > [nodemon] starting `node server.js`
```

```
[0] Server running on http://localhost:3001
[1] > Vite v7.3.1 ready in 1410 (any) ms
[1] Local:   http://localhost:3000/
```

Option B — Separately

Terminal 1:

```
npm run dev:backend
```

Terminal 2:

```
npm run dev:frontend
```

Service URLs

Service	URL	Description
Frontend	http://localhost:3000	Main application — open in browser
Backend API	http://localhost:3001	REST API base URL
Health Check	http://localhost:3001/health	Verify API is running

> Warning: Always start the backend before or alongside the frontend. Without the API running, all pages will appear empty.

6. API Documentation

Base URL: http://localhost:3001

All request and response bodies use JSON. Include the Content-Type: application/json header on all POST and PUT requests.

6.1 Endpoints Overview

Method	Endpoint	Description
GET	/health	Check if the API server is running
GET	/transactions	Get all transactions (supports query filters)
GET	/transactions/:id	Get a single transaction by UUID
POST	/transactions	Create a new transaction

6.2 Transaction Object

Field	Type	Required	Description	Example
id	string (UUID v4)	Auto-generated	Unique identifier — never include in requests	5efd04ee-f04f-...
transaction_date	string	Yes	Date in YYYY-MM-DD format	2026-03-15
account_number	string	Yes	Format: XXXX-XXXX-XXXX	5551-2345-6789
account_holder_name	string	Yes	Full name of account holder	Jennifer Lee
amount	number	Yes	Positive number	175.25
status	string	Yes	Pending, Settled, or Failed	Pending

6.3 GET /transactions

Returns all transactions. Supports optional query filters:

Parameter	Example	Description
status	?status=Pending	Filter by Pending, Settled, or Failed
search	?search=jennifer	Search by account holder name
from	?from=2026-01-01	Transactions on or after this date
to	?to=2026-03-31	Transactions on or before this date

```
curl http://localhost:3001/transactions
curl "http://localhost:3001/transactions?status=Pending&search=jennifer"
```

Success response — 200 OK:

```
[{ "id": "5efd04ee-...", "transaction_date": "2026-03-07",
   "account_number": "5551-2345-6789", "account_holder_name": "Jennifer Lee",
   "amount": 175.25, "status": "Pending" }]
```

6.4 POST /transactions

Creates a new transaction. Do not include the id field — it is auto-generated.

```
curl -X POST http://localhost:3001/transactions
  -H "Content-Type: application/json"
  -d '{"transaction_date":"2026-03-15","account_number":"5551-0000-1234",
       "account_holder_name":"John Doe","amount":500.00,"status":"Pending"}'
```

Success — 201 Created:

```
{ "id": "a3f9c2d1-...", "transaction_date": "2026-03-15", ... }
```

Error — 400 Bad Request:

```
{ "errors": ["Account number must be in format XXXX-XXXX-XXXX."] }
```

7. Validation

Validation runs at two levels — frontend for instant UX feedback, and backend as the true security gate. Both use the same rules from the shared @transaction/shared package. If a rule changes, updating shared/validation.js is all that is needed.

Field	Rule	Error Message
transaction_date	Required	Transaction date is required.
account_number	Required, format XXXX-XXXX-XXXX	Account number must be in format XXXX-XXXX-XXXX.
account_holder_name	Required	Account holder name is required.
amount	Required, positive number	Amount must be a positive number.
status	Required, Pending / Settled / Failed	Invalid status. Must be: Pending, Settled, or Failed.

8. Testing

8.1 Verify API is Running

```
curl http://localhost:3001/health
```

Expected: { "status": "ok", "timestamp": "2026-03-15T10:00:00.000Z" }

8.2 Partial CRUD Test Flow

Run these in order to test every operation end-to-end:

1. Create:
```
curl -X POST http://localhost:3001/transactions
  -H "Content-Type: application/json"
  -d '{"transaction_date":"2026-03-15","account_number":"5551-0000-1234",
       "account_holder_name":"John Doe","amount":500.00,"status":"Pending"}'
```
2. Get all (copy the id from step 1):
```
curl http://localhost:3001/transactions
```
3. Get one:
```
curl http://localhost:3001/transactions/<id>
```

8.3 Test Validation

These should return 400 errors:

```
# Missing fields
curl -X POST http://localhost:3001/transactions -H "Content-Type: application/json"
-d '{"amount":100}'

# Bad account number format
curl -X POST http://localhost:3001/transactions -H "Content-Type: application/json"
  -d '{"transaction_date":"2026-03-15","account_number":"12345",
       "account_holder_name":"Test","amount":100,"status":"Pending"}'
```

8.4 Useful curl Flags

Flag	Usage	What it Does
-i	curl -i http://localhost:3001/transactions	Shows HTTP status code and headers
-s | python3 -m json.tool	curl -s .../transactions | python3 -m json.tool	Pretty-prints JSON response

9. System Architecture

9.1 Monorepo Structure

Package	npm Name	Role
shared/	@transaction/shared	Shared validation logic — consumed by backend and frontend
backend/	backend	Express REST API, CSV operations, migration
frontend/	frontend	React, Vite, transaction UI

9.2 Request Flow

```
Browser
  |
  v
React Frontend (port 3000)
  |  fetch('/api/transactions')
  v
Vite Proxy  -- rewrites /api --> http://localhost:3001
  |
  v
Express API (port 3001)
  |  readTransactions() / writeTransactions()
```

```
  v
transactions.csv
```

9.3 Shared Validation Flow

```
shared/validation.js
  |
  +-- backend/server.js                validates on every POST/PUT (blocks bad data)
  |
  +-- frontend/Transactions.jsx        validates before API call (instant feedback)
```

9.4 Branch Strategy

Branch	Purpose	Rule
main	Production-ready code	Never commit directly — merge from staging only
staging	Integration testing	Merge backend + frontend here, test, then merge to main
backend	Backend development	Merge into staging when complete
frontend	Frontend development	Merge into staging when complete
shared	Shared package changes	Merge into backend and frontend before staging

9.5 File Structure

```
FC-transaction-management-system/
├── package.json                 root monorepo + scripts
├── node_modules/                hoisted dependencies
├── shared/
|   ├── package.json             name: @transaction/shared
|   └── validation.js            shared validation (ESM)
|   ├── transactions.sample.csv  sample data (committed)
├── backend/
|   ├── server.js                Express API, all endpoints
|   ├── migrate.js               one-time UUID migration
|   ├── transactions.csv         live data (git-ignored)
|   └── package.json
└── frontend/
    ├── src/
    |   ├── pages/
    |   |   └── Transactions.jsx CR(UD) table
    |   ├── App.jsx              layout, routing, sidebar
    |   └── api.js               fetch helpers
    ├── vite.config.js           port + /api proxy
    └── package.json
```

10. Troubleshooting

Error	Cause	Fix
curl: (7) Failed to connect port 3001	Backend not running	Run npm run dev:backend and check terminal for errors
'vite' is not recognized	node_modules out of sync	Delete node_modules, run npm install from root
'nodemon' is not recognized	nodemon not installed	Run: npm install nodemon --save-dev --workspace=backend
Transactions not displaying	Field name mismatch (CSV vs API)	Check readTransactions() maps CSV headers to snake_case
Invalid hook call (React)	Duplicate React instances	Delete all node_modules, run npm install from root only
Package subpath not exported	Wrong shared import path	Use: import { validateTransaction } from '@transaction/shared'
npm workspaces not found	npm version older than v7	Run: npm install -g npm
Records lost after POST	readTransactions vs writeTransactions key mismatch	Use readRawTransactions() inside POST route